AprilTag Tracking

1. Function Description

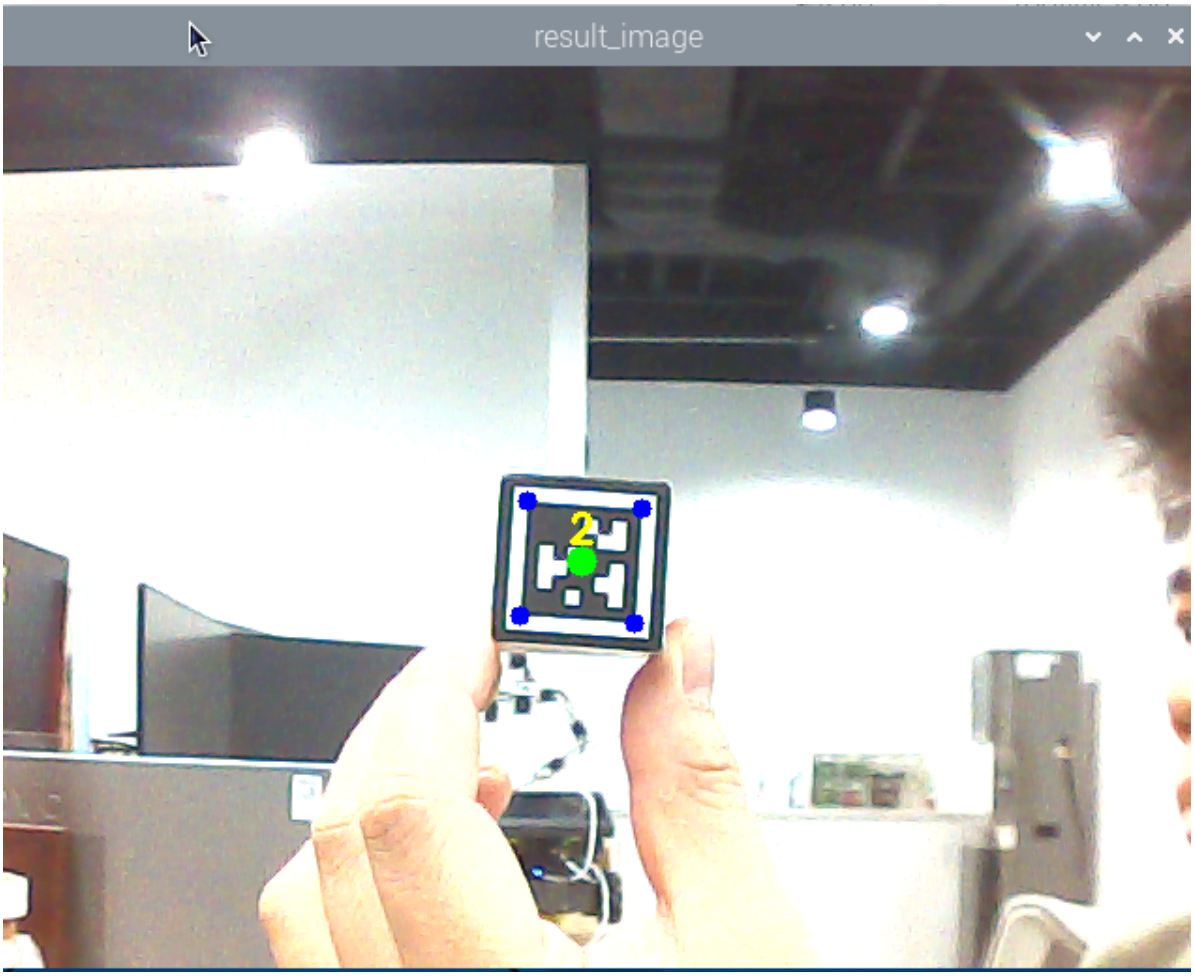
After the program runs, the robotic arm will lift up and enter tracking mode. When an AprilTag appears in the camera's field of view, the robotic arm will detect and adjust its posture to keep the AprilTag within the center range. Slowly move the AprilTag, and the robotic arm will follow the AprilTag's movement.

2. Function Startup

After entering the docker container, enter the following commands in the docker terminal:

```
# Start USB camera node and inverse kinematics node
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start AprilTag tracking program node
ros2 run dofbot_follow apriltag_follow_2D
```

After the program runs successfully, the robotic arm will move to its initial posture. Hold the AprilTag and place it within the robotic arm's camera field of view. Slowly move the AprilTag, and the robotic arm will follow the AprilTag's movement, adjusting its posture to keep the AprilTag within the center range of the image, as shown in the figure below:



3. Code Analysis

3.1. apriltag_follow_2D

After entering the docker container, the source code path is located at:

```
/root/dofbot_ws/src/dofbot_follow/dofbot_follow/apriltag_follow_2D.py
```

Import necessary library files:

```
import cv2
from dofbot_sorting.vutils import draw_tags
from dt_apriltags import Detector
from cv_bridge import CvBridge
import cv2 as cv
import time
from rclpy.node import Node
import rclpy
from sensor_msgs.msg import Image
import threading
# Custom PID calculation library, located at
/root/dofbot_ws/src/dofbot_follow/dofbot_follow/PID.py
import dofbot_follow.PID as PID
from Arm_Lib import Arm_Device
import os
```

Create robotic arm control object and control the robotic arm to move to gesture recognition posture:

```
self.Arm = Arm_Device()
self.init_joints = [90, 150, 0, 20, 90, 0]
self.Arm.Arm_serial_servo_write6_array(self.init_joints,2000)
```

Initialize PID:

```
self.xservo_pid = PID.PositionalPID(0.5, 0.02, 0.1)
self.yservo_pid = PID.PositionalPID(0.5, 0.02, 0.1)
```

Create subscription to image topic:

```
self.subscription =
self.create_subscription(Image,'/image_raw',self.ImageCallback,1)
```

Image topic callback function ImageCallback:

```
def ImageCallback(self,color_frame):
    # Convert frame to opencv format
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame,'rgb8')

    tags = self.at_detector.detect(cv2.cvtColor(rgb_image, cv2.COLOR_RGB2GRAY),
False, None, 0.025)
    tags = sorted(tags, key=lambda tag: tag.tag_id)
    draw_tags(rgb_image, tags, corners_color=(0, 0, 255), center_color=(0, 255,
0))
```

```
    frame1 = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2BGR)
    cv2.imshow("result_image", frame1)
    key = cv2.waitKey(1)
    if len(tags) > 0 :
        #print("tag: ",tags)
        cur_id = tags[0].tag_id
        # Get the center coordinates of the AprilTag
        center_x, center_y = tags[0].center
        # If the AprilTag's center coordinates x or y are not within the center
range, execute XY_track function with the current AprilTag center coordinates as
parameters
        if (abs(center_x-320) >30 or abs(center_y-240)>30) :
            self.XY_track(center_x,center_y)
            print("Tracking")
            print("-------------------------------------")
```